

Practice programs

1)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[100], n, key, choice;
```

```
    int i, low, high, mid, found = 0;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter %d elements:\n", n);
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    printf("\nEnter element to search: ");
```

```
    scanf("%d", &key);
```

```
    printf("\nChoose Search Method:\n");
```

```
    printf("1. Linear Search\n");
```

```
    printf("2. Binary Search\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
switch(choice)
{
    case 1: // Linear Search
        for(i = 0; i < n; i++)
        {
            if(arr[i] == key)
            {
                printf("Element found at position %d\n", i + 1);
                found = 1;
                break;
            }
        }
        if(found == 0)
        {
            printf("Element not found\n");
        }
        break;
    case 2: // Binary Search (Array must be sorted)
        low = 0;
        high = n - 1;
        while(low <= high)
        {
            mid = (low + high) / 2;
            if(arr[mid] == key)
```

```

    {
        printf("Element found at position %d\n", mid + 1);
        found = 1;
        break;
    }
    else if(arr[mid] < key)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}
if(found == 0)
{
    printf("Element not found\n");
}
break;

default:
    printf("Invalid choice\n");
}
return 0;
}

```

```
C:\Users\Devi\Desktop\nitya(1) x + v
Enter number of elements: 2
Enter 2 elements:
11
24

Enter element to search: 11

Choose Search Method:
1. Linear Search
2. Binary Search
Enter your choice: 2
Element found at position 1

Process returned 0 (0x0)   execution time : 15.880 s
Press any key to continue.
|
```

```
C:\Users\Devi\Desktop\nitya(1) x + v
Enter number of elements: 3
Enter 3 elements:
12
14
19

Enter element to search: 14

Choose Search Method:
1. Linear Search
2. Binary Search
Enter your choice: 1
Element found at position 2

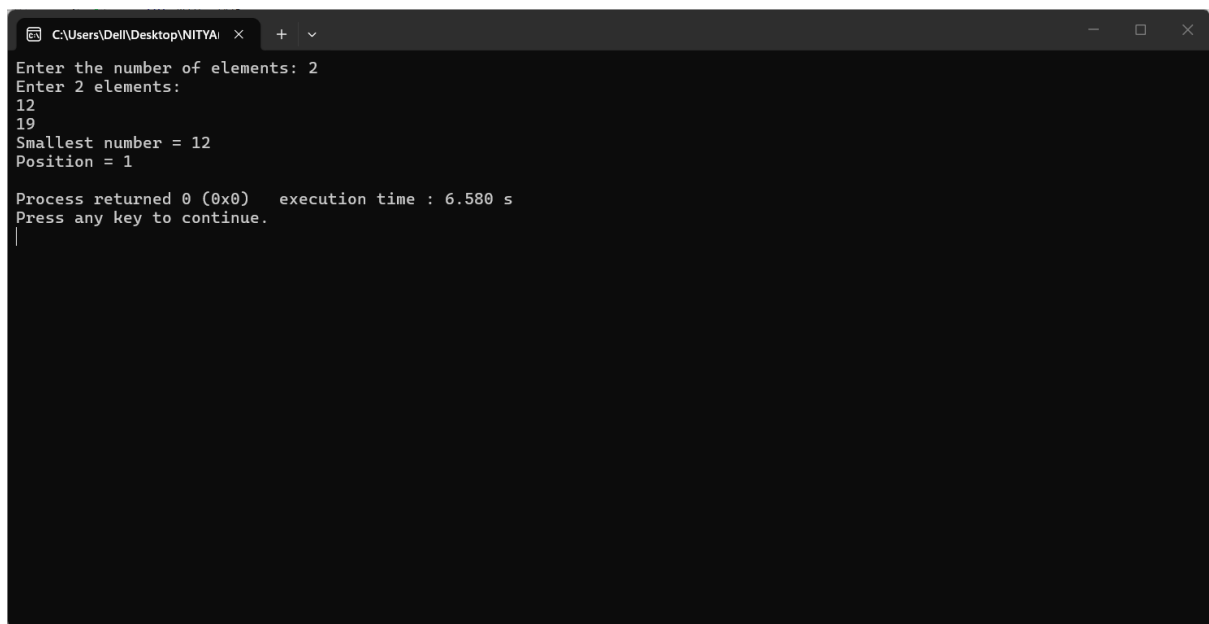
Process returned 0 (0x0)   execution time : 30.689 s
Press any key to continue.
|
```

2)

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int arr[100], n, i;  
    int min, pos;  
    printf("Enter the number of elements: ");  
    scanf("%d", &n);  
    printf("Enter %d elements:\n", n);  
    for(i = 0; i < n; i++)  
    {  
        scanf("%d", &arr[i]);  
    }  
    min = arr[0];  
    pos = 0;  
  
    for(i = 1; i < n; i++)  
    {  
        if(arr[i] < min)  
        {  
            min = arr[i];  
            pos = i;  
        }  
    }  
    printf("Smallest number = %d\n", min);  
    printf("Position = %d\n", pos + 1);  
    return 0;  
}
```



```
C:\Users\Dell\Desktop\NITYA > Enter the number of elements: 2
Enter 2 elements:
12
19
Smallest number = 12
Position = 1

Process returned 0 (0x0)   execution time : 6.580 s
Press any key to continue.
|
```

3)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr[100], n, i, j, found = 0;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter %d elements:\n", n);
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
for(j = i + 1; j < n; j++)  
{  
    if(arr[i] == arr[j])  
    {  
        printf("Duplicate element found: %d\n", arr[i]);  
        found = 1;  
        break;  
    }  
}  
if(found == 1)  
    break;  
}  
if(found == 0)  
{  
    printf("No duplicate elements found in the array.\n");  
}  
return 0;  
}
```

```
C:\Users\Devi\Desktop\NITYA' x + v
Enter number of elements: 3
Enter 3 elements:
12
16
18
No duplicate elements found in the array.

Process returned 0 (0x0)   execution time : 29.085 s
Press any key to continue.
```

LAB PROGRAM

```
#include <stdio.h>

int main()
{
    int n,i,j;

    int at[20], bt[20], ct[20], tat[20], wt[20], p[20];

    float avg_tat=0, avg_wt=0;

    int temp;

    printf("Enter number of processes: ");

    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        p[i]=i+1;

        printf("\nProcess %d\n",i+1);

        printf("Arrival Time: ");
```



```

scanf("%d",&at[i]);

printf("Burst Time: ");

scanf("%d",&bt[i]);

}

/* Sorting according to Arrival Time */
for(i=0;i<n;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(at[i] > at[j])
        {
            temp=at[i]; at[i]=at[j]; at[j]=temp;

            temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;

            temp=p[i]; p[i]=p[j]; p[j]=temp;

        }
    }
}

ct[0] = at[0] + bt[0];

for(i=1;i<n;i++)
{
    if(ct[i-1] < at[i])

        ct[i] = at[i] + bt[i];

    else

        ct[i] = ct[i-1] + bt[i];

```

```

}

for(i=0;i<n;i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    avg_tat += tat[i];
    avg_wt += wt[i];
}

printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
}

avg_tat /= n;
avg_wt /= n;

printf("\nAverage Turnaround Time = %.2f",avg_tat);
printf("\nAverage Waiting Time = %.2f",avg_wt);

return 0;
}

```

```
C:\Users\Devi\Desktop\NITYA | x + v
Enter number of processes: 3

Process 1
Arrival Time: 22
Burst Time: 21

Process 2
Arrival Time: 12
Burst Time: 24

Process 3
Arrival Time: 11
Burst Time: 15

p    AT    BT    CT    TAT    WT
3     11     15     26     15     0
2     12     24     50     38     14
1     22     21     71     49     28

Average Turnaround Time = 34.00
Average Waiting Time = 14.00
Process returned 0 (0x0)  execution time : 37.709 s
Press any key to continue.
|
```